



UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Facultad de Ingeniería

Object-Oriented Programming

SYNKRO

Presented by:

Kevin Navarrete

Valeria Usma

Instructor:

Carlos Andrés Sierra

1. Introduction

Instant communication is a necessity in today's world. This project seeks to create an efficient application that enables secure and accessible user interaction, inspired by the basic functions of Skype. The application will allow users to register, log in, send private and group messages, share files, manage contacts, and customize their profile.

The project seeks to offer a practical and functional communication solution while applying the fundamental concepts of object-oriented programming. The main motivation is to design a flexible, organized, and scalable system with well-structured classes and relationships that facilitate future growth and maintenance.

2. Objectives

The project aims to develop an application that allows users to:

- Send and receive messages.
- Make voice and video calls.
- Implement an authentication system so that each user has a personal account.
- Create an intuitive and easy-to-use interface.

Specific Objectives

- Implement a secure authentication system.
- Develop private and group messaging functionality.
- Manage public and private groups.
- Allow contact management and profile customization.
- Apply SOLID principles of object-oriented programming.
- Model the system with cohesive classes and properly structured relationships.
- Implement a flexible, scalable, and maintainable design.

3. Requirements

Functional Requirements

1. Registration and login using email and password.
2. Sending and receiving private messages between users.
3. Creating and managing public and private groups.
4. Contact management and profile customization.
5. Account control: temporary suspension and deletion.
6. Invitation system for friends and groups. Making voice calls. Envío y recepción de mensajes privados entre usuarios.
7. Creation and management of public and private groups.
8. Contact management and profile customization.
9. Account control: temporary suspension and deletion.
10. Invitation system for friends and groups. Making voice calls.

Non-Functional Requirements

1. Protection of user data and communications.

2. Prevention of duplicate usernames and email addresses.
3. Immediate application of account changes.
4. Support for multiple users without affecting platform stability. Prevención de duplicidad de nombres de usuario y correos electrónicos.

Although the model includes voice and video calling functionality, this feature will not be implemented in the current version of the project due to time and scope constraints. The system design allows for the possibility of adding this functionality in future versions, ensuring that its integration is compatible with the current architecture.

4. User Stories

Story: User Registration (Priority: High)

User Story: As a new user, I want to register with an email, a unique username, and a password.

Acceptance Criteria: The user must enter valid and unique data. The system displays an error if the email or name already exists. The user is automatically logged in after registration.

Story: Private Communication (Priority: High)

User Story: As a user, I want to send messages with other users.

Acceptance Criteria: Privacy and encryption must be guaranteed. Messages are only accessible to participants.

Story: Group Messaging (Priority: High)

User Story: As a user, I want to send messages to groups and receive instant replies.

Acceptance Criteria: Messages are displayed instantly with names and timestamps. Participants can respond immediately.

Story: Group Management (Priority: High)

User Story: As a user, I want to create or delete groups.

Acceptance Criteria: Only the creator or administrators can delete groups. Deletion clears all group history.

Story: Member Management (Priority: High)

User Story: As a user, I want to add or remove members from a group I manage.

Acceptance Criteria: Confirmation is required when deleting members. Administrators can manage the participant list.

Story: Temporary Suspension (Priority: High)

User Story: As a user, I want to be able to temporarily close my account.

Acceptance Criteria: The deactivated account is invisible to other users and can be reactivated upon login.

Story: Joining Groups (Priority: Medium)

User Story: As a user, I want to join existing groups.

Acceptance Criteria: Users can search for and join public groups, or request access to private groups.

Story: Creating Groups (Priority: Medium)

User Story: As a user, I want to create public or private groups.

Acceptance Criteria: The creator decides the group's privacy and becomes the initial administrator.

Story: Group Management (Priority: Medium)

User Story: As a user, I want only administrators to manage groups.

Acceptance Criteria: The administrator can assign and revoke permissions to other users.

Story: Interface Customization (Priority: Low)

User Story: As a user, I want to customize the app's appearance.

Acceptance Criteria: Users can switch between light/dark modes, modify backgrounds, and font sizes. Changes are automatically saved.

Story: Voice Calls (Priority: Low)

User Story: As a user, I want to make voice calls with my contacts to communicate directly, but this feature will be considered for future versions due to the current scope of the project.

5. Mock-Ups

6. CRC Cards

1. Class: User

Responsibilities:

- Manage profile and contacts.
- Participate in conversations.
- Upload files.
- Make calls.

Contributors:

- Authentication Service
 - Message Sender
 - Call
 - Call History
 - File
-

2. Class: Authentication Service

Responsibilities:

- Register users.
- Manage logins and logouts.

Contributors:

- User
-

Interface: Message Sender

Responsibilities:

- Send messages.

Contributors:

- Conversation
 - User
-

3. Class: Conversation

Responsibilities:

- Manage messages.
- Store participants.

Collaborators:

- Message
 - File
 - Channel
 - Direct Message
-

4. Class: Channel

Responsibilities:

- Manage public or private groups.
- Add or remove participants.

Collaborators:

- Conversation
-

5. Class: Direct-Message

Responsibilities:

- Manage one-on-one chats.
- Block or unblock users.

Collaborators:

- Conversation
-

6. Class: Message

Responsibilities:

- Store text or files.
- Edit and delete messages.

Contributors:

- File
 - Conversation
-

7. Clase: File

Responsibilities:

- Manage files.
- Allow downloads.

Contributors:

- Image
 - Doc
 - User
 - Conversation
-

8. Clase: Call

Responsibilities:

- Initiate and end voice calls.
- Calculate call duration.

Contributors:

- User
 - Call History
-

9. Class: Call History

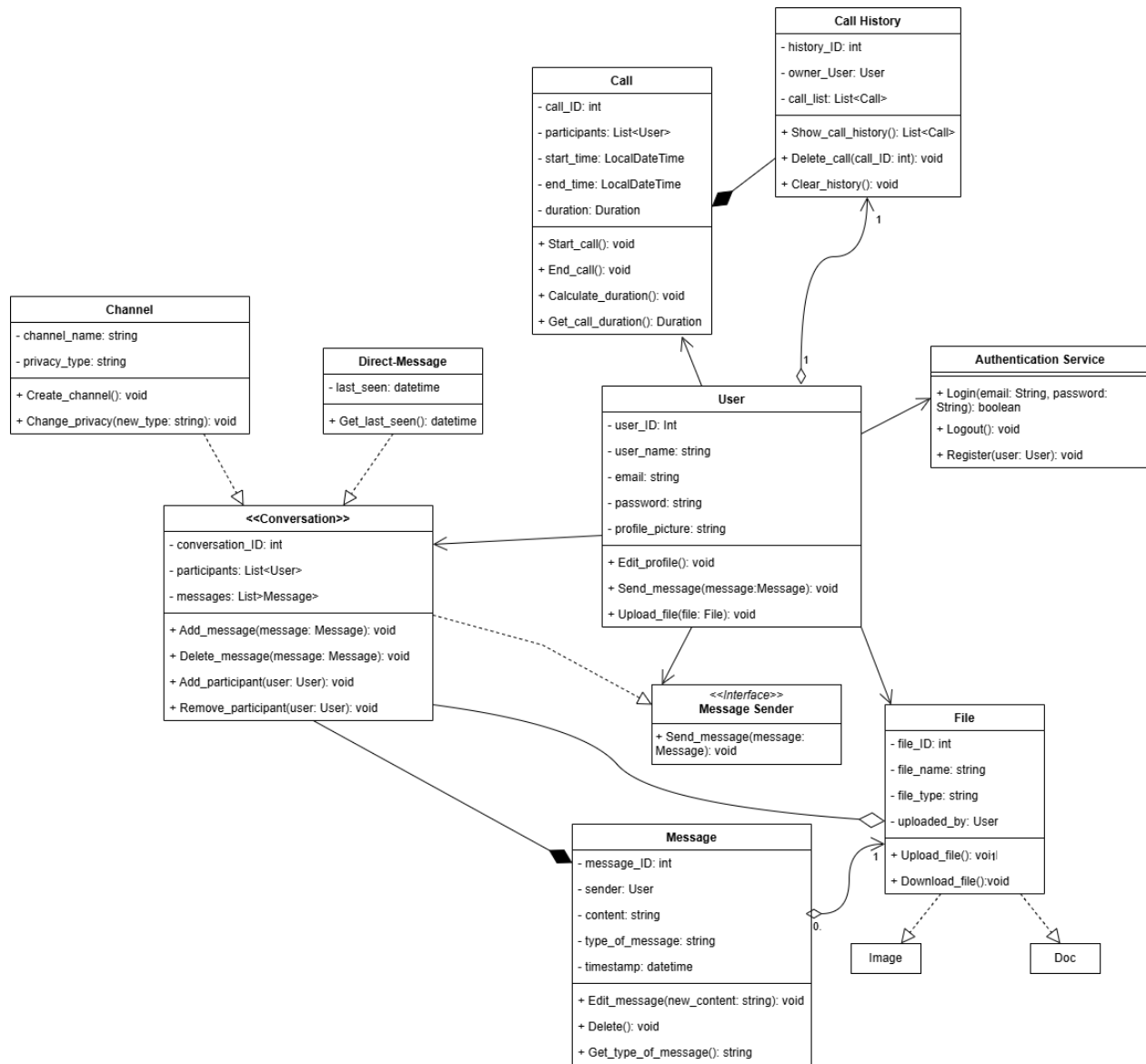
Responsibilities:

- Record user call history.
- Delete calls.

Contributors:

- Call
- User

7. UML Diagram



Main modeling decisions:

1. **User:** Manages the profile, contacts, and calls. It does not handle authentication directly.
2. **Authentication Service:** Exclusive class for registration, login, and logout.
3. **Conversation:** General class that organizes chats.
4. **Channel and DirectMessage:** Subclasses of Conversation, allowing differentiation between group and private chats.
5. **Message:** An object that represents each message; it can contain text, files, or both.
6. **File:** Parent class for different file types (Image, Doc). It allows extending the system without modifying it.
7. **Call and CallHistory:** Model of voice calls and their history per user.

8. Message Sender: Interface that ensures message sending without being directly coupled to the conversation type.

The AuthenticationService class was created so that authentication could be managed independently. The User class is limited to storing profile data and managing contacts, ensuring that each class has a single responsibility.

Furthermore, the system allows adding new file types (for example, images or documents) without modifying existing classes. File is an extensible class, which facilitates future extensions.

Furthermore, the Channel and DirectMessage classes are subtypes of Conversation and can be used where a conversation is expected without affecting the system logic.

Finally, the MessageSender interface was created so that classes that send messages are not forced to implement methods that do not correspond to them. User only interacts with this interface and does not depend on specific implementations.

The User class depends on the IMessageSender abstraction, which allows changing the class that manages message sending without affecting the User structure. This decoupling ensures flexibility in the system.

8. How do we apply OOP concepts

For the project implementation, each class was designed to represent real-world objects, such as users, messages, groups, calls, and files. The relationships between classes were modeled to reflect their interactions and dependencies, allowing each object to maintain its own state and behaviors.

Encapsulation: The attributes of each class were declared private to protect data integrity, and can only be accessed or modified through public methods (getters and setters).

Inheritance: Hierarchies were created, such as the Conversation class, which is general and specializes in Channel and DirectMessage, allowing reuse of common code and behaviors.

Polymorphism: Thanks to inheritance and the implementation of the IMessageSender interface, classes can be used interchangeably, allowing for flexible handling of different types of conversations. Composition and Aggregation: Composition relationships were established, such as Conversation and Message, where messages do not exist outside the conversation, and aggregation relationships, such as the association between Conversation and File, where files can exist independently.

Abstraction: Classes such as File and interfaces such as IMessageSender were created to encapsulate general behaviors, allowing other classes to interact with them without knowing the internal details of their implementation..

9. Code